Reviewer Pack — Minimal Symmetric Function Rank and Communication Complexity…

Entry c43ced097cbb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 13:08:17 UTC

Minimal Symmetric Function Rank and Communication Complexity Rank Inequality

Entry ID: `c43ced097cbb` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 13:08:17 UTC

1. Conjecture

Field A (mathematical branch): Symmetric Function Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every k-communication protocol P, the minimal symmetric function rank (srank(P)) of its associated matrix representation is linearly correlated with its communication complexity rank (crank(P)), such that srank(P) = Ω(crank(P)).

Rationale (proposer's reasoning):

> Symmetric functions provide a rich algebraic structure to encode polynomial properties. This conjecture explores the potential of symmetric function theory in capturing nontrivial aspects of communication complexity, which is known to be inherently related to polynomial representations.

Taxonomy category: `symmetric_function_ranks` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `621ceecc6de408ac`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the seeds produce symmetric function rank values that are greater than or equal to 0.8 times their respective communication complexity ranks, with an average symmetric function rank less than or equal to 3 across all seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"symmetric function rank" AND "communication complexity rank" AND inequality"` - `"matrix representation" IN Symmetric Function Theory AND crank(P)"` - `srank(P) >= Omega(crank(P))`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        factor = matrix[i][i]
        for j in range(i, n):
            matrix[i][j] /= factor

        for k in range(n):
            if k != i:
                factor = matrix[k][i]
                for j in range(i, n):
                    matrix[k][j] -= factor * matrix[i][j]

    return matrix

def srank(matrix):
    reduced_matrix = gaussian_elimination(matrix)
    rank = sum(1 for row in reduced_matrix if any(row))
    return rank

def crank(protocol):
    # Placeholder for communication complexity calculation
```

```python
    # For simplicity, assume it's a function of the protocol size
    return len(protocol)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        protocol = [random.randint(0, 1) for _ in range(n)]
        srank_value = srank(protocol)
        crank_value = crank(protocol)

        if crank_value == 0:
            continue

        correlation = srank_value / crank_value
        results.append({
            "n": n,
            "srank_value": srank_value,
            "crank_value": crank_value,
            "correlation": correlation
        })

    if not results:
        return {
            "metric_name": "Correlation",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No valid instances found"
        }

    mean_correlation = sum(result["correlation"] for result in results) / len(results)
    max_n = max(result["n"] for result in results)

    return {
        "metric_name": "Correlation",
        "metric_value": mean_correlation,
        "instances_tested": len(results),
        "n_max": max_n,
        "conjecture_holds": mean_correlation >= 0.8 and mean_correlation <= 3,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
```

```
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"]
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["metric_value"] is not None for result in results):
        RESULT = "SUPPORTED" if support_fraction >= 0.8 else "FALSIFIED"
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        RESULT = f"FALSIFIED counterexample=\"correlation_out_of_bounds\" first_failing_seed={firs
    else:
        RESULT = "INCONCLUSIVE no_valid_instances"

    print(f"RESULT: {RESULT} mean={mean_metric_value} std=0 support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e30f03.py", line 102, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e30f03.py", line 59, in run_trial
    srank_value = srank(protocol)
                  ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e30f03.py", line 42, in srank
    reduced_matrix = gaussian_elimination(matrix)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_21e30f03.py", line 24, in gaussian_elim
    if abs(matrix[k][i]) > abs(matrix[max_row][i]):
           ~~~~~~~~~~^^^
TypeError: 'int' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14271
2	preregistration	ollama_remote	glm4:latest	0	0	11824
3	novelty	ollama_remote	glm4:latest	0	0	9404
4	novelty	ollama_remote	glm4:latest	0	0	9789
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13369
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10060
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9642
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10340
9	judge	ollama_remote	glm4:latest	0	0	26582

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 115280 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c43ced097cbb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c43ced097cbb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c43ced097cbb.tar.gz` (if generated)